



Prompt Engineering ■ Agents de coding

Agents de coding & Prompt Engineering

Comprendre les LLM, puis savoir les piloter par le prompt

Des LLM aux agents qui codent — exemples Dev & DevOps

Objectifs

- Situer l'**IA générative** vs discriminative.
- Comprendre un **LLM**, puis un **agent de coding**.
- Acquérir un **prompting** reproductible.
- Cadrer un usage **sûr et reproductible**.

Déroulé

1. IA discriminative vs IA générative
2. Des LLM aux **agents de coding**
3. Techniques de prompting appliquées
4. Risques, sécurité & bonnes pratiques
5. Atelier & quizz

Fil rouge

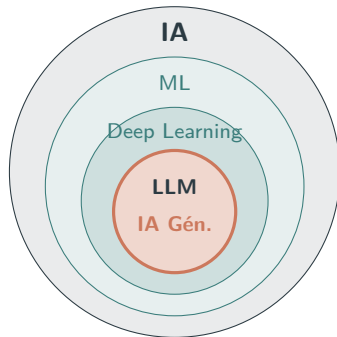
Un LLM **prédit du texte plausible** — il ne « comprend » pas votre projet. La qualité dépend de votre **prompt**, du **contexte** et des **outils** fournis.

IA discriminative vs IA générative



Où se situe le LLM dans le paysage de l'IA ?

- **IA** — faire exécuter à une machine des tâches qui demandent de l'intelligence.
- **Machine Learning** — apprendre des *données* plutôt que coder des *règles*.
- **Deep Learning** — ML à base de réseaux de neurones profonds.
- **NLP** — traitement automatique du langage.
- **LLM** — NLP à large échelle, fondé sur les *Transformers*.
- **IA générative** — *produit* du contenu (texte, code, config) au lieu de seulement classifier.



Les trois grands types d'apprentissage

Supervisé

Apprend de (X, y) : on connaît la **réponse attendue**.

- Classification (build OK / KO)
- Régression (latence prédite)

SVM, Random Forest, XGBoost, réseaux de neurones.

Non supervisé

Apprend de X seul : aucune étiquette, on cherche une **structure**.

- Clustering (familles de logs)
- Détection d'anomalies

K-means, DBSCAN, autoencodeurs.

Par renforcement

Un **agent** agit, reçoit une **récompense**, ajuste sa politique.

- Jeux, robotique
- RLHF (alignement des LLM)

Q-learning, PPO, DQN.

Cas particulier : auto-supervisé

Les LLM s'entraînent en **prédissant le token suivant** à partir de texte brut (dont du *code*) — pas d'étiquette manuelle, mais une supervision extraite automatiquement du corpus.

Discriminative — $P(y | x)$

- Apprend une **frontière** de décision.
- Entrée $x \rightarrow$ étiquette y .
- Ex. : ce log est-il anormal ? cette PR est-elle risquée ?
- Outils : régression logistique, SVM, Random Forest, XGBoost, CNN.

Générative — $P(x)$ ou $P(x, y)$

- Apprend la **distribution** des données.
- Peut **échantillonner** / produire du nouveau contenu.
- Ex. : écrire ce test, ce Dockerfile, cette doc.
- Outils : Transformers, modèles de diffusion, VAE / GAN.

À retenir

Les LLM sont **génératifs** (ils produisent le token le plus probable) mais savent aussi *classifier, extraire, résumer*. Les deux familles sont **complémentaires**.

Cas d'usage — IA discriminative (vue d'ensemble)

Dev & qualité de code

- Classification de tickets / bugs (routage).
- Détection de **secrets** commités, de duplication.
- NER sur stack traces (extraction d'entités).
- Détection de code *smell* / vulnérabilités (SAST).

DevOps / CI-CD

- **Scoring de risque** d'une PR / d'un déploiement.
- Prédiction de tests *flaky*.
- Classification des échecs de build.
- Scan d'images : classer les findings par criticité.

Point commun : une *réponse fermée* (une classe, un score) à partir d'une entrée — entraînée sur des données étiquetées. **Détail dans la slide suivante.**

Dev & qualité de code

- **Classification tickets / bugs** — router un ticket vers la bonne équipe / catégorie.
Ex. : « crash au login » → bug/auth, équipe Identité.
- **Détection secrets / duplication** — repérer clés & tokens poussés par erreur, ou du code copié-collé.
Ex. : ghp_... dans un commit → build bloqué.
- **NER sur stack traces** — étiqueter exception, classe, fichier :ligne.
Ex. : NullPointerException at OrderService.java:42.

DevOps / CI-CD

- **Scoring de risque d'une PR** — prédire qu'un changement casse la prod (taille du diff, zones touchées, historique).
Ex. : 30 fichiers cœur modifiés → risque élevé → revue renforcée.
- **Tests flaky** — repérer les tests instables (échouent sans changement de code).
Ex. : échoue 1 fois sur 5 → mis en quarantaine.
- **Classification des échecs de build** — catégoriser la cause.
Ex. : log → « dépendance manquante » → notifie l'owner.

Cas d'usage — IA générative (vue d'ensemble)

Dev

- Génération / refactor de code, **tests**, doc.
- Revue de PR, explication de legacy (héritage).
- Génération de SQL depuis le langage naturel.
- **RAG** : Q&R sur la doc / le code interne.

DevOps / CI-CD

- Pipelines (GitHub Actions, GitLab CI).
- **Dockerfile**, scripts de release.
- Génération de manifests Kubernetes, chart Helm.
- **Agents** qui *agissent* (cf. partie 2).

Point commun : une *sortie ouverte* (du texte, du code, de la config) — qu'il faut toujours **relire et tester**. **Détail dans la slide suivante.**

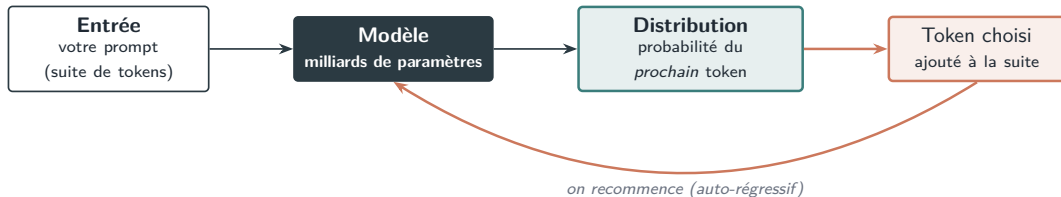
Dev

- **Génération / refactor, tests, doc** — produire ou réécrire du code.
Ex. : « écris les tests unitaires de cette classe ».
- **Revue de PR / explication de legacy** — commenter un diff, expliquer du vieux code.
Ex. : « que fait cette fonction de 200 lignes ? » → résumé.
- **SQL depuis le langage naturel** — traduire une question en requête.
Ex. : « top 5 clients par CA » → `SELECT ... ORDER BY ... LIMIT 5.`

DevOps / CI-CD

- **Pipelines CI/CD** — générer la config d'un pipeline.
Ex. : « GitHub Actions : build, test, déploie sur push main ».
- **Dockerfile / scripts de release** — conteneuriser, automatiser la livraison.
Ex. : Dockerfile multi-stage pour une app .NET.
- **Manifests Kubernetes** — générer Deployment / Service / Ingress.
Ex. : « expose ce service, port 8080, 3 replicas ».

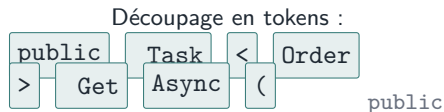
Qu'est-ce qu'un LLM, concrètement ?



- Le modèle **prédit le token suivant** le plus probable, encore et encore.
- Il n'exécute pas de code, ne « sait » pas si c'est vrai : il produit du **texte plausible**.
- Conséquence directe : il peut être **brillant** et **se tromper avec aplomb**.

Tokens & fenêtre de contexte

- Un **token** $\approx$ un morceau de mot (~ 4 caractères en anglais, souvent moins en code).
- La **fenêtre de contexte** = tout ce que le modèle « voit » en une fois (prompt + historique + fichiers joints).
- Au-delà, il **oublie** : d'où l'intérêt de compacter et de cibler.
- **Coût** et **latence** croissent avec le nombre de tokens.



Task<Order> GetAsync(
→ **8 tokens** environ.

Ordre de grandeur : un gros fichier source = plusieurs milliers de tokens. La fenêtre n'est pas infinie.

Non déterministe

- Même prompt → réponses **différentes**.
- La **température** règle la créativité / variabilité.
- Pour du code/config : on veut généralement **peu de hasard** + des contraintes fortes.

Hallucination

Le modèle invente une réponse **plausible mais fausse** :

- une méthode d'API qui n'existe pas ;
- un flag docker / kubectl inexistant ;
- un paquet NuGet/Maven/PyPI fantôme.

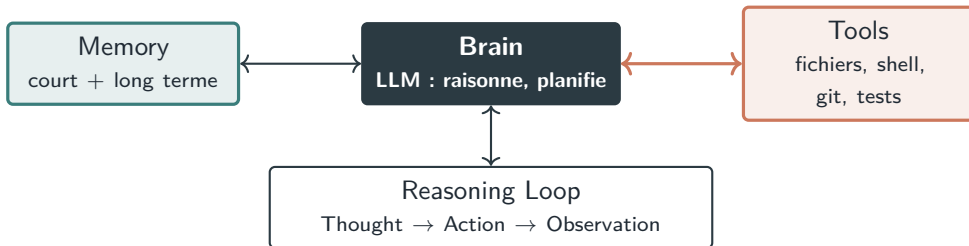
Réflexe ingénieur

Toujours **compiler, tester, valider** (build, tests, dry-run). L'IA **propose**, l'ingénieur **valide**. Ne jamais merger / appliquer du contenu IA non vérifié.

Des LLM aux agents de coding



Du LLM « nu » à l'agent

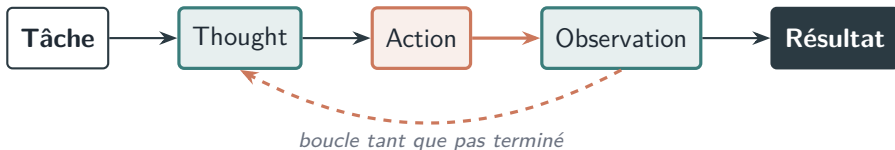


LLM seul vs agent

Un LLM **seul** génère du texte, une seule fois. Un **agent** ajoute (1) une **mémoire** pour persister le contexte, (2) des **outils** pour interagir avec le monde réel (lire des fichiers, lancer des commandes), (3) une **boucle** pour raisonner sur plusieurs tours et *réagir au résultat*.

Le pattern ReAct : Reasoning + Acting

Le LLM alterne explicitement **Thought** (réflexion), **Action** (appel d'outil) et **Observation** (résultat réel). Chaque tour informe le suivant — base de tous les agents de coding.



Pourquoi c'est puissant pour le code

L'agent *voit le résultat réel* (sortie de compilation, test rouge, log) — il ne se contente pas d'« opiner ». S'il se trompe, il **le constate** et change de stratégie. Garde-fou indispensable : `max_iter`.

Ce qu'il sait faire

- **Lire** le dépôt : fichiers, arborescence, grep.
- **Éditer** des fichiers (code, YAML, Dockerfile).
- **Exécuter** : build, tests, docker, git.
- **Lire** la sortie / les logs et **itérer**.

Outils typiques

- Système de fichiers (read / write / search).
- Shell sandboxé (commandes autorisées).
- Git, gestionnaire de paquets.
- Tests, linters, build / dry-run.

Exemples du marché

Claude Code (notre fil rouge des jours 2–4), Cursor, GitHub Copilot Agent, Cline. . . Tous reposent sur la même mécanique : un LLM + des outils + une boucle ReAct.

Trace d'un agent de coding (exemple)

Tâche confiée à l'agent : *le pipeline CI échoue sur un test du service Catalog.*

Thought: Je lis le test et l'endpoint concernés avant de toucher au code.

Action: `read_file("Catalog.Api/Endpoints/CatalogEndpoints.cs")`

Observation: GET /catalog/{id} fait 'products[id]' -> indexation directe (off-by-one).

Thought: Je reproduis l'échec en local avant de corriger.

Action: `run_shell("dotnet test --filter GetById_returns_404")`

Observation: 1 failed -- renvoie 200 au lieu de 404 pour un id absent.

Thought: La cause est l'indexation. Je passe à un lookup par Id + 404, puis je relance.

Action: `edit_file(...)` ; `run_shell("dotnet test --filter GetById")`

Observation: 2 passed.

Final: Correctif appliqué (FirstOrDefault par Id, 404 si absent). Diff prêt pour revue.

Même boucle pour un manifeste K8s qui ne démarre pas ou un pipeline CI à réparer.

Agent autonome vs workflow balisé

Critère	Agent autonome	Workflow balisé
Qui décide ?	le LLM à chaque tour	le dev fixe les étapes
Prévisibilité	faible (émergent)	élevée (explicite)
Bon pour	exploration, prototypage	production, processus tracé
Exemple	explorer un repo, débbugger	triage de tickets, release

Recommandation (Anthropic, *Building Effective Agents*)

« Construire le workflow le plus simple qui fonctionne ; ne passer aux agents que quand c'est nécessaire. » En pratique : démarrer en agent **ReAct** pour itérer vite, puis **figer** les chemins critiques.

Agents de coding — cas d'usage par métier

Dev

- Implémenter une feature de bout en bout (code + tests).
- Corriger un bug : reproduire → patcher → vérifier.
- Migration guidée (framework, version de langage).
- Générer des tests & expliquer du code legacy.

DevOps / CI-CD

- Écrire / réparer un pipeline qui échoue.
- Générer un Dockerfile multi-stage et le tester.
- Corriger un manifeste K8s, valider en dry-run.
- Packager un chart Helm, `helm lint / template`.

Gouvernance — l'humain reste aux commandes

L'agent **propose** des actions ; les actions sensibles (merge, apply, prod) passent par une **validation humaine** et une **traçabilité**. On verra les *permissions* de Claude Code plus loin dans la formation.

Techniques de prompting appliquées



Prompt vague

ecris un script de deploiement

Résultat : générique, outil au hasard, pas idempotent, pas de gestion d'erreur.

Prompt cadré

Role : ingénieur DevOps senior.
Tache : script de deploiement bleu/vert.
Contexte : K8s 1.29, Helm 3, registre GHCR, rollback automatique si la sonde readiness echoue.
Format : script bash + commentaires.

Résultat : ciblé, idempotent, conforme à votre stack.

Principe

Le modèle comble les trous que vous laissez. **Moins vous précisez, plus il invente.**

Anatomie d'un bon prompt

Rôle — « Tu es un ingénieur DevOps senior. . . »

Tâche — l'action précise et unique à réaliser

Contexte — stack, versions, conventions, fichiers concernés

Format — code seul / diff / JSON / étapes

Contraintes — perfs, sécurité, « ne touche pas à X »

Exemples (optionnel) — montrer le style attendu

Moyen mnémotechnique : **R-T-C** + Format + Contraintes. Tout n'est pas toujours nécessaire — mais Rôle/Tâche/Contexte presque toujours.

Rôle · Tâche · Contexte — la base, R-T-C (Dev)

Donner un **rôle** oriente le style ; une **tâche** unique évite le hors-sujet ; le **contexte** ancre dans *votre* projet.

Role : Tu es un developpeur Spring Boot senior, expert tests.

Tache : Ecris les tests d'integration du OrderController.

Contexte : Spring Boot 3, JUnit 5 + MockMvc, base PostgreSQL via
Testcontainers. Le controller expose POST /orders et GET /orders/{id}.

Conventions du projet : un fichier *IT.java par controller, given/when/then.

Format : un seul fichier Java, compilable, sans commentaire superflu.

Le modèle sait maintenant *qui* il est, *quoi* produire et *dans quel cadre* — il invente beaucoup moins.

Zero-shot — le réflexe par défaut (Dev)

Demander **directement**, sans exemple : on s'appuie sur ce que le modèle a déjà appris. **À tester en premier** (rapide, peu de tokens).

```
Classe ce log parmi : BUILD | TEST | APPLICATIF | DEPENDANCE.
```

```
Reponds par un seul mot, sans explication.
```

```
Log : "OutOfMemoryError: Java heap space at OrderService.process(...)"
```

Si la sortie dérive (jargon interne, cas limites, vocabulaire métier) → passez au **few-shot** (slide suivante).

Few-shot — montrer des exemples (DevOps)

Plutôt que *décrire* une convention, **montrez-la** avec 1 à 3 exemples. Idéal pour aligner sur vos standards internes.

Voici notre convention de nommage et de tag des images Docker :

```
ghcr.io/acme/catalog-api:1.4.2-prod
```

```
ghcr.io/acme/ordering-api:2.0.0-preprod
```

Format : `ghcr.io/<org>/<service>:<version>-<env>`

Genere les commandes 'docker build' et 'docker tag' pour le service payment-api en version 1.0.0 destinee a la preprod, en respectant EXACTEMENT cette convention :

Le « few-shot » aligne le modèle sur vos conventions sans avoir à les énumérer en prose.

Balises XML / délimiteurs — séparer consigne & données (DevOps)

Encapsuler les données dans des **balises** : le modèle ne confond plus la *consigne* avec le *contenu*. Réduit aussi le risque d'**injection de prompt**.

```
<role>Tu es ingénieur DevOps.</role>
<instruction>Resume l'échec du déploiement et donne la cause racine probable.</
  instruction>
<logs>
03:12 ERROR readiness probe failed: connection refused on :8080
03:12 WARN  rollout stuck: 0/3 pods ready, image pull OK
</logs>
```

Tout ce qui vient d'une source externe (logs, ticket, page web) va dans une balise → traité comme *donnée*, jamais comme instruction.

Imposer le format de sortie (DevOps)

Cadrer la **forme** de la réponse la rend parsable, collable, applicable.

Diff prêt à appliquer

```
Corrige ce manifeste K8s  
(limits memoire manquantes).  
Reponds UNIQUEMENT par un  
diff git unifie, sans texte.
```

Sortie structurée

```
Analyse ces logs et renvoie  
du JSON :  
{ "cause": "...",  
  "service": "...",  
  "remediation": "..."} }
```

Très utile pour automatiser ensuite : revue de PR, scripts, intégration *headless* de Claude Code dans un pipeline.

Raisonnement guidé — *chain-of-thought* (DevOps)

Pour le **diagnostic**, demandez au modèle de **raisonner par étapes** *avant* de conclure.

```
Ce pod est en CrashLoopBackOff par intermittence. Procède par étapes :  
1. Resume ce que disent les events et les logs ci-dessous.  
2. Liste les hypotheses (OOMKilled ? sonde liveness trop stricte ?  
   dependance indisponible au demarrage ?).  
3. Indique la cause racine la plus probable et comment la confirmer.  
4. SEULEMENT ENSUITE, propose le correctif minimal (diff du manifeste).
```

Pourquoi ça marche

Forcer les étapes intermédiaires réduit les raccourcis erronés — le modèle « pose le problème » avant de répondre.

Décomposer — plan avant action / prompt chaining (DevOps)

Sur une grosse tâche, demandez d'abord un **plan** que vous validez, puis l'exécution étape par étape.

N'ecris pas encore de code.

Donne-moi un plan en 5 étapes pour migrer ce déploiement de docker-compose vers des manifestes Kubernetes (Deployment, Service, ConfigMap, Ingress, sondes). Pour chaque étape : risque et test de validation (kubectl dry-run / smoke test).

J'approuverai le plan avant que tu gènes l'étape 1.

Vous gardez le contrôle, vous évitez un gros bloc ingérable, et vous pouvez itérer étape par étape.

Self-Consistency — voter entre plusieurs réponses (décision)

Poser la **même question plusieurs fois** (température > 0) et garder la réponse **majoritaire**. Coûte $\times N$ appels → réservé aux **décisions à fort enjeu**.

```
Ce correctif corrige-t-il COMPLETEMENT la faille ? Reponds OUI/NON + raison.  
Contexte : patch qui echappe l'input SQL mais pas la sortie ecrite en logs...  
  
# on execute 5 fois -> verdict majoritaire (ici 4/5 = NON, fuite en logs)
```

Usage : go/no-go de déploiement, validation d'un correctif de sécurité, choix d'une stratégie de rollback.

Tree of Thoughts — comparer plusieurs options (archi)

Demander plusieurs **stratégies en parallèle**, les **noter** selon des critères explicites, puis recommander.

Propose 3 strategies pour fiabiliser ce traitement batch (idempotence, reprise sur erreur). Pour chacune :

- principe (2 lignes)
- cout / complexite (note 1-5)
- risque principal

Termine par une recommandation argumentee.

Pour les décisions d'architecture où il faut peser des options : file d'attente vs polling, blue/green vs canary, SQL vs NoSQL.

Contexte technique & contraintes (DevOps)

Les **versions** et **conventions** évitent les hallucinations d'API et la config « hors projet ».

Sans contexte

Optimise ce Dockerfile.

Peut proposer des instructions dépréciées, une mauvaise image de base, etc.

Avec contraintes

Optimise ce Dockerfile.

Contraintes : base node:20-alpine,
build multi-stage, utilisateur
non-root, aucune instruction
deprecie, garde l'ENTRYPOINT.

Astuce : un bon **CLAUDE.md** (vu plus loin) fournit ce contexte *automatiquement* à chaque prompt.

Itérer chirurgicalement

Ne jetez pas le résultat : **corrigez par petites touches** en pointant précisément le problème.

Bien, mais ce Dockerfile invalide le cache a chaque build : le 'COPY . .' est place avant 'dotnet restore'. Corrige UNIQUEMENT ce point (copie les *.csproj puis restore, AVANT de copier le reste). Ne touche pas au reste du Dockerfile.

- Donnez le **message d'erreur** exact, l'**entrée** qui casse, le **comportement attendu**.
- Délimitez le **périmètre** (« ne touche pas à X ») pour éviter les régressions.

Indice de confiance — faire expliciter l'incertitude (prod)

Demander un **score de confiance** par sortie → on **route selon un seuil** : automatique / à vérifier / escalade humaine.

Extrais {service, severite, cause} de cet echec de pipeline.

Pour chaque champ : une confiance 0-100% + une justification courte.

Si confiance < 70%, marque le champ "A_VERIFIER".

Echec : "build catalog -- step=docker push, exit=1, 'unauthorized: token expired'"

Routage typique : ≥ 90 % auto · 70–90 % « à vérifier » · < 70 % escalade humaine. Attention : les LLM peuvent être **sur-confiants** — calibrer.

Active-Prompt — amélioration itérative sur cas réels

Repérer les cas où le prompt **échoue** → ajouter des **exemples ciblés** → retester → itérer jusqu'au taux visé.

```
Classifieur de tickets -- progression mesuree sur un jeu reel :  
Cycle 1 (zero-shot) : 60% corrects  
Cycle 2 (+ few-shot sur "RAS", "a voir") : 80%  
Cycle 3 (+ regle de tri explicite) : 95%  
Cycle 4 (+ gestion des cas limites) : 98% -> on fige le prompt
```

Traiter un prompt comme du code : on le fait **progresser** sur un jeu d'exemples représentatif, puis on **versionne** la version retenue.

Récap (1/2) — base & structuration

Technique	Quand l'utiliser
Zero-shot	Tâche courante, prototypage rapide.
Few-shot	Imposer une convention / un format (naming, commits, Dockerfile).
R-T-C (= role prompting)	Presque toujours : ancrer rôle, tâche, contexte projet.
Balises XML	Séparer consigne & données, réduire l'injection.
Format imposé	Sortie à parser ou appliquer : diff, JSON, étapes.
Contexte & contraintes	Éviter les hallucinations d'API / versions.
Itérer	Affiner une sortie sans tout réécrire (périmètre borné).

Récap (2/2) — raisonnement, décision & production

Technique	Quand l'utiliser
Chain-of-thought	Diagnostic, debug, analyse de logs multi-étapes.
Décomposer (= prompt chaining)	Grosse tâche : migration, refonte, gros refactor.
Self-Consistency	Décision critique : go/no-go, correctif de sécurité.
Tree of Thoughts	Choix d'architecture : comparer plusieurs options.
Indice de confiance	Extraction / classif en prod : seuil + escalade humaine.
Active-Prompt	Amélioration continue d'un prompt sur cas réels.
ReAct	Agents avec outils (shell, tests, RAG...) — voir partie 2.

Les techniques se **combinent** : rôle + balises XML + format JSON + contraintes de version dans un même prompt.

Pièges fréquents (anti-patterns)

À éviter

- Prompt vague, sans rôle ni contexte.
- **Tout demander d'un coup** (code + tests + doc + config).
- Omettre les **versions** / conventions.
- Mélanger **instruction** et **données** à traiter.
- Accepter la sortie **sans relire ni tester**.

À faire

- Une **tâche** claire à la fois.
- Fournir le **contexte** (stack, fichiers, contraintes).
- Demander le **format** utile (diff, JSON, étapes).
- **Itérer** par petites corrections ciblées.
- **Compiler / planifier / tester** systématiquement.

Risques, sécurité & bonnes pratiques



Confidentialité — ce qui ne va jamais dans un prompt

Autorisé

- Code / config **anonymisés** ou de sandbox
- Documentation publique
- Messages d'erreur génériques
- Extraits sans donnée réelle

Interdit

- **Secrets** : clés, mots de passe, *connection strings*
- **Données personnelles** / PII
- **Code propriétaire** sensible
- Noms d'hôtes, IP, identifiants internes

Réflexe

En cas de doute : **anonymiser** (noms, IP, identifiants) avant d'envoyer. Préférer les modes **Enterprise** / **Business** (pas d'entraînement sur vos données).

Données & propriété

- **Données personnelles** / PII : hors des prompts (RGPD).
- **Secrets & credentials** : jamais en clair.
- **Propriété intellectuelle** : attention au code généré et à ses licences.

Risques techniques de l'IA

- **Hallucinations** (API, attributs, comportements).
- **Dépendances suggérées** : paquets inexistants ou vulnérables (*slopsquatting*).
- **Prompt injection** via données externes (logs, tickets).

Vers une charte interne

Définir ensemble : quels outils, quels modes, quelles données, qui valide quoi, et comment tracer l'usage de l'IA.

Atelier & synthèse



Atelier — bibliothèque de prompts Dev & DevOps

Par binôme, sur le code et la CI / les conteneurs du sandbox : produire 5 prompts réutilisables, testés, capitalisés dans le dépôt Git partagé.

1. Générer
endpoint /
manifeste

2. Revue
bugs, sécurité

3. Diagnostic
logs / bug

4. Tests
cas limites

5. Refactor
code / config

Chaque prompt applique les techniques vues : R-T-C, format imposé, contraintes de version, few-shot pour le style.

Atelier — gabarit de prompt « Tests / Validation »

Exemple de gabarit paramétrable que les binômes adapteront et valideront :

```
Role : ingénieur {Dev | DevOps} senior.  
Tache : écris les tests / la validation de {cible : methode | manifeste |  
    Dockerfile}.  
Contexte : {stack + versions}, {xUnit | JUnit | kubectl dry-run | docker build},  
    conventions du projet : {given/when/then, 1 fichier par cible...}.  
Couvrir : cas nominal, cas limites, valeurs nulles / erreurs attendues.  
Format : un seul livrable applicable, sans explication hors du code.  
Contrainte : n'invente aucune API / aucun attribut ; si une dépendance manque,  
    signale-le au lieu de l'inventer.
```

Critères de validation

Une bibliothèque de **5 prompts** par binôme, exécutés sur du code / des conteneurs réels, + une charte d'usage esquissée.

IA & LLM

- Discriminatif = classer ;
génératif = produire.
- Le LLM prédit du **texte plausible**, ne « comprend » pas.
- Peut **halluciner** :
l'humain valide.

Agents

- LLM + **outils** + **boucle**
ReAct.
- L'agent de coding lit,
édite, exécute, itère.
- Garde-fous &
validation humaine.

Prompt

- **R-T-C** + format +
contraintes.
- **Few-shot, CoT, plan**,
itération.
- Jamais de données
sensibles.

La suite — Claude Code

On applique tout ça dans **Claude Code** : le contexte (**CLAUDE.md**), les permissions, et l'agent qui *agit* sur le code et la CI / les conteneurs.

1. Différence entre IA discriminative et IA générative ? Un exemple de chaque côté Dev/DevOps.
2. Qu'ajoute un *agent* par rapport à un LLM seul ?
3. Décrivez la boucle ReAct (Thought / Action / Observation).
4. Citez les 3 éléments quasi toujours nécessaires dans un prompt.
5. Pour un diagnostic (debug / logs), pourquoi demander un raisonnement *avant* la conclusion ?
6. Donnez 2 informations à ne jamais mettre dans un prompt.



Capitalisation : déposez vos meilleurs prompts dans le dépôt Git partagé de l'équipe.

Merci

Cap sur Claude Code.



Bassem Ben Hamed